

Funktioner, pekare och poster

- Mer om funktioner, returvärden och parametrar
- Hur organiserar man program?
- Program i flera filer
- Pekare och fält
- Mer om textsträngar
- Dynamisk minneshantering

Funktionsdefinition

- En funktion *definieras* med huvud och kropp
 - huvudet säger hur funktionen skall anropas
 - kroppen säger vad funktionen gör

```
float medelv (float x, float y)
{
    return ((x + y) / 2);
}
```

Funktionsdeklaration, prototyp

- För att anropa en funktion räcker det att veta hur den skall anropas, d.v.s. ha sett huvudet
 - ett ensamt huvud kallas *deklaration* eller *prototyp*

```
float medelv (float x, float y); /* huvudet med semikolon */  
  
int main()  
{  
    float f=4.5; int i=14;  
    f = medelv(f,i);  
    printf("%f", f);  
}
```

Några detaljer

- ▶ En prototyp behöver inte ha med namnen på parametrarna `float medelv (float, float);`
- ▶ Innan ett anrop skall man ha sett definition eller prototyp
- ▶ Definitionen kan finnas i en fil, anropet i en annan, då måste en prototyp finnas där anropet görs
- ▶ Det får finnas många prototyper, men bara en definition

Parametrar

- Parametrar skickas *alltid* som *kopior* av värden i C

```
float medelv(float, float);
```

```
float f=3.1, g=4.3, h;
```

```
h = medelv(f,g);
```

```
/* utan att studera medelv vet vi alltid:
```

```
   f och g ändras inte av anropet */
```

Parametrar (forts.)

- Parametrar fungerar som variabler som fått ett initialt värde av anroparen

```
float medelv (float x, float y)
{
    float ff;
    ff = (x + y) / 2;
    x = 45678;    /* påverkar bara det lokala x */
    return ff;
}
```

Rekursion

- Eftersom parametervariabler är färska för varje anrop kan en funktion anropa en ny upplaga av sig själv för att lösa ett delproblem
 - skolboksexemplet är fakultet: $n! = 1*2*3*...*n$

```
int fakultet (int n) {  
    if (n < 2)  
        return 1;  
    else  
        return fakultet(n-1) * n;  
}
```

Synlighet (eng. *scope*)

- Identifierare syns olika långt
 - variabler och parametrar inne i funktioner syns bara inuti funktionen.
 - funktioner och variabler utanför funktioner syns i resten av filen, och i andra filer om de deklarerats där
 - variabler utanför funktioner, *globala variabler*, kan påverkas av alla funktioner
 - det ger svåröverblickbara program, var maximalt sparsam med globala variabler

Lagringsklasser

- Variabler är `auto` eller `static`
 - lokala variabler och parametrar är standardmässigt `auto`, d.v.s. skapas automatiskt när funktionen anropas och dör när funktionen avslutas
 - globala variabler är `static`, d.v.s. skapas och initieras innan `main` anropas och finns i där under hela programkörningen
 - lokala variabler kan deklareras `static`, kommer då ihåg sitt värde från anrop till nästa anrop, och syns bara inuti funktionen,

const - som inte får ändras

► Data kan deklareras const

- då kollar kompilatorn att ingen kod försöker ändra detta

```
const float pi = 3.1415926;
```

```
const int vuxen 28;
```

```
vuxen = 17;  /* kompileringsfel */
```

Ordning i stora program

- Stora program delas upp i flera filer
 - det är praktiskt och skapar ordning
 - uppdelningen sker så logiskt som möjligt
 - varje fil innehåller funktioner och variabler som hör ihop, kallas ibland *modul* eller *paket*
 - filnamnet kan lämpligtvis uttrycka modulens uppgift, t ex `queue.c`
 - de deklARATIONER och prototyper en användare av köer behöver samlas då i filen `queue.h`
 - den som vill använda kömodulen gör `#include "queue.h"`

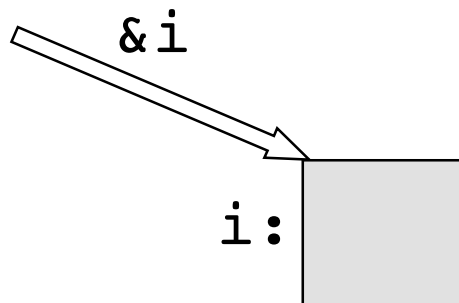
Pekare

- Pekare är värden som refererar till andra objekt
 - tänk på dem som pilar! (egentligen är de minnesadresser)
 - en pekar-variabel kan innehålla sådana pil-värden
 - en pekarvariabel är noga med att peka ut bara en viss typ av objekt, t ex en `int`-pekare pekar bara på `int`

&-operatorn

- Givet en variabel-”låda” får man en pekare till den med operatorn &

```
int i;
```

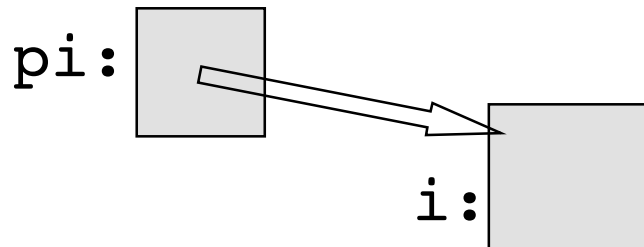


Deklaration av pekare

- En pekar-variabel deklarerar med den typ den skall kunna peka på

- pekare deklarerar med ”pekarmärket” *

```
int i;      /* i kan innehålla heltal, men är oinitierad */  
int *pi;    /* pi kan peka på heltal, men är oinitierad */  
i = 48;     /* i får ett värde */  
pi = &i;    /* pi får ett värde */
```



/* PS. Vi har ännu inte sagt ett ord
om vad pekare skall vara bra för */

* –operatorn

- Givet en pekare, får man det utpekade med operatorn *

```
int i,j;    int *pi;

pi = &i;

*pi = 567;   /* det utpekade (dvs i) får värdet 567 */

j = *pi;     /* det utpekades värde (dvs i:s värde)
               ges till j */

printf("%d", *pi);

/* om pi är en heltalspekare, används *pi precis som
ett heltal */
```

Varför använda pekare?

- Pekare används i C för tre huvudsyften
 - skicka som parametrar, så att det utpekade kan ändras inifrån funktionen
 - stega igenom fält av effektivitetsskäl
 - används som del av `struct`:er (nästa kapitel) för att bygga länkade strukturer

Pekare som parameter

- Om en pekare skickas som parameter, kan funktionen via pekaren ändra på det utpekade
 - detta är mycket vanligt i biblioteksfunktioner, t ex `scanf`

```
void dubblera(int *pp)
{
    *pp = *pp * 2;
}

int main()
{
    int i;
    i = 17;
    dubblera(&i);      /* dubblera(i) vore alldeles fel */
    printf("%d\n", i)
}
```

Pekar-aritmetik och fält

- Man kan addera pekare och heltal (!)
 - resultatet är en ny pekare ett antal ”snäpp” längre fram

```
int iflt[10];
```

```
int *pi, *pp;
```

```
pi = &iflt[0]; /* pi pekar nu på 0:te elementet i iflt */
```

```
pp = pi + 3; /* pp pekar tre heltalssnäpp längre fram */
```

```
*pp = 568; /* iflt[3] får värdet 568 */
```

```
pi++; /* flytta fram pi ett heltalssnäpp */
```

```
*pi = 876; /* iflt[1] får värdet 876 */
```

Ett ensamt fältnamn

- Ett ensamt fältnamn är *nästan* alltid en pekare till 0:te elementet i fältet (undantag `sizeof iflt`)
 - d.v.s `iflt` är oftast samma sak som `&iflt[0]`
 - den andra `for`-satsen kan gå lite fortare, åtminstone tror många C-programmerare det.

```
int i; int iflt[10]; int *pi;
for (i = 0; i < 10; i++)
    printf("%d\n", iflt[i]);
for (pi = iflt; pi < iflt + 10; pi++)
    printf("%d\n", *pi);
```

Vad är indexering egentligen?

```
int iflt[10];
```

```
int *ip;
```

```
ip = iflt;
```

Då gäller:

```
ip + 3 == iflt + 3
```

```
*(ip + 3) == *(iflt + 3) == iflt[3] == ip[3]
```

Alltså gäller alltid: $x[y] == *(x + y)$

- ▶ Indexering *endast* på pekarvärden
- ▶ Indexering kan göras på *alla* pekarvärden
- ▶ Oberoende var de kommer ifrån:
 - fältnamn, pekarvariabel, m m

Fält som parameter

- Om ett fält skickas som parameter kommer bara en pekare fram
 - då kan inte den anropade funktionen ta reda på storleken

```
int iflt[35]; int s;  
/* iflt får värden */
```

```
s = summera35(iflt); /* inte så lyckat */  
s = summera(iflt, 35) /* vi skickar med storleken */
```

Fält som parameter

```
#include <stdio.h>

int summera(int *pi, int n)  /* summera fältets element */
{
    int i, sum = 0;
    for(i = 0; i < n; i++)
        sum += pi[i];        /* indexering går bra på pekare */
    return sum;
}
```

Initiering av fält

- Ett helt fält kan initieras, men inte tilldelas som helhet

```
int iflt1[7] = { 3, 4, -5, 7, 3, 9, -23 };  
int iflt2[100] = { 1, 4 };    /* 98 odefinierade värden */  
int iflt3[] = { 55, 66, 3, 4, 9, 1, 3 }; /* lagom stort */  
  
iflt1 = { 6, 5, 6, 2, 5, 1, 1 }; /* FEL! */  
iflt1 = iflt3; /* FEL! */
```

Textsträngar och pekare

- Textsträngar lagras i teckenfält
 - värdet av en teckensträng är alltså en teckenpekare
 - den pekar ut första tecknet i strängen

```
char cflt[4] = "Hej";  
char *cp;  
cp = cflt;  
printf("Hopp"); /* alla tre är char-pekare */  
printf(cflt);  
printf(cp);
```

Textsträng som parameter

```
#include <stdio.h>

void printstr(char *s)
{
    while(*s != '\0')
    {
        putchar(*s);    /* kan slås ihop till */
        s++;            /* putchar(*s++);      */
    }
}

int main()
{
    printstr("Hej hopp!\n");
}
```

Dynamiskt minne

- Två fel på vanliga fält med fast storlek
 - för små
 - onödigt stora
- Vanliga fält måste deklareraras med konstant storlek

```
void funk(int n)
{
    int iflt[n]; /* FEL!   (men infört i C99) */
    /* ... */
}
```

Dynamiskt minne (forts.)

- Färskt minne kan *allokeras* med `malloc`
 - parameter: antal bytes
 - returvärde: pekare till dessa
 - lämna tillbaka minne med `free`

```
void funk(int n)
{
    int *pi;
    pi = malloc(n * sizeof(int));
    /* pi kan nu användas
       som om det vore ett n stort heltalsfält */
    free(pi); /* bara minne man fått från malloc! */
}
```

Fält av fält

- Det går bra att deklarera fält med flera dimensioner
 - Nackdel som med enkla fält: fast inflexibel storlek som måste vara känd vid kompileringen

```
int sales_per_day[12][31]; /* data för hela året */  
                           /* alla månader lika långa! */  
  
sales_per_day[1][9] = 457;
```

Fält av fält dynamiskt

- Vi skaffar en pekare till ett fält av pekare

```
int **sales_per_day;

sales_per_day = (int**) malloc(12*sizeof(int*));

sales_per_day[0] = (int *) malloc(31*sizeof(int)); /*jan*/
sales_per_day[1] =
    (int *) malloc((28+leapyear())*sizeof(int)); /*feb*/

/*etc*/

sales_per_day[1][9] = 457; /* likadant som för fält! */

/* varje malloc behöver en free så småningom */
```

Parametrar till `main`

- ▶ Vi har hittills sett `main` utan parametrar
 - den kan ha noll eller två
 - värdet av dessa är text(ord) som skrevs på kommandoraden när programmet startades
 - om programmet startades med `./a.out bull 45 strut` blir `argc` 4 och de fyra strängorden var sin sträng (`char *`) i ett fält som utpekas av `argv`.

```
int main(int argc; char **argv)
{
    int i;
    for (i = 1; i < argc; i++) {
        printf("%s ", argv[i]); /* skriv ut orden */
    }
}
```